

開始使用 Web Serial API

來源：<https://codelabs.developers.google.com/web-serial?hl=zh-tw>

步驟數：9

在本程式碼研究室中，您將建構一個與 BBC micro:bit 遊戲板互動的網頁，以便在 5x5 LED 顯示螢幕上顯示圖片。您將瞭解 Web Serial API，以及如何使用可讀取、可寫入和轉換串流，透過瀏覽器與序列裝置通訊。

目錄

1. [1. 簡介](#)
2. [2. 關於 Web Serial API](#)
3. [3. 開始設定](#)
4. [4. 開啟序列連線](#)
5. [5. 控制 LED 矩陣](#)
6. [6. 連接 micro:bit 按鈕](#)
7. [7. 使用轉換串流剖析傳入資料](#)
8. [8. 關閉序列埠](#)
9. [9. 恭喜](#)

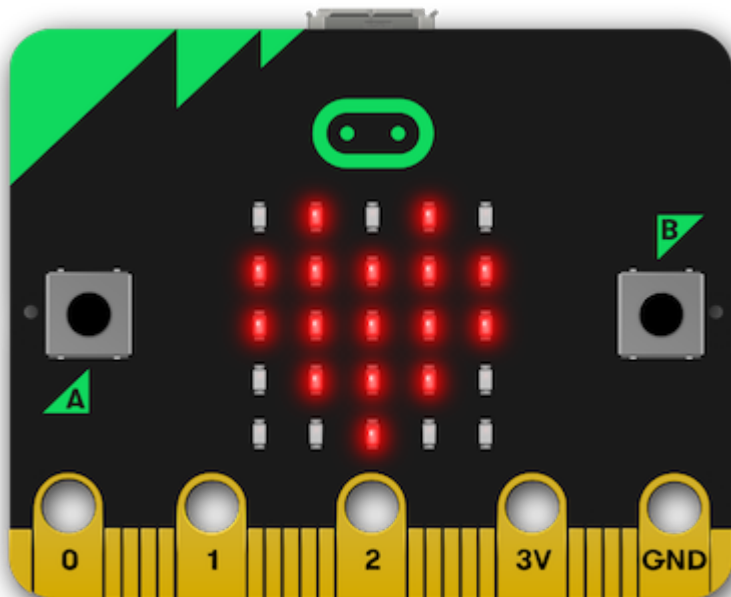
步驟 1 · 約 0 分鐘

1. 簡介

上次更新時間：2022 年 9 月 19 日

建構項目

在本程式碼研究室中，您將建構一個網頁，使用 Web Serial API 與 [BBC micro:bit](#) 開發板互動，在 5x5 LED 矩陣上顯示圖片。您將瞭解 Web Serial API，以及如何使用可讀取、可寫入和轉換的串流，透過瀏覽器與序列裝置通訊。



課程內容

- 如何開啟及關閉 Web Serial 序列埠
- 如何使用讀取迴圈處理輸入串流中的資料
- 如何透過寫入串流傳送資料

軟硬體需求

- 搭載 [Espruino 韌體 2v04](#) 的 [BBC micro:bit v1](#) 開發板
- 新版 Chrome (80 以上版本)
- 瞭解 HTML、CSS、JavaScript 和 Chrome 開發人員工具

我們選擇使用 micro:bit v1 進行本程式碼研究室，是因為它價格實惠，提供幾個輸入 (按鈕) 和輸出 (5x5 LED 顯示螢幕)，而且可以提供額外的輸入和輸出。如要進一步瞭解 micro:bit 的功能，請參閱 [Espruino 網站的 BBC micro:bit 頁面](#)。

2. 關於 Web Serial API

[Web Serial API](#) 可讓網站透過指令碼讀取及寫入序列裝置。這項 API 可讓網站與微控制器和 3D 印表機等序列裝置通訊，在網路和實體世界之間建立橋樑。

許多控制軟體都是使用網頁技術建構而成，例如：

- [Arduino Create](#)
- [Betaflight Configurator](#)
- [Espruino IDE](#)
- [MakeCode](#)

在某些情況下，這些網站會透過使用者手動安裝的原生代理程式應用程式與裝置通訊。在其他情況下，應用程式會透過 Electron 等架構，以封裝的原生應用程式形式提供。在其他情況下，使用者必須執行額外步驟，例如使用 USB 隨身碟將編譯的應用程式複製到裝置。

讓網站與控制的裝置直接通訊，可提升使用者體驗。

步驟 3 · 約 1 分鐘

3. 開始設定

取得程式碼

我們已將本程式碼研究室所需的資料都放到 Glitch 專案中。

1. 開啟新的瀏覽器分頁，然後前往 <https://web-serial-codelab-start.glitch.me/>。
 2. 按一下「Remix Glitch」連結，建立自己的入門專案版本。
 3. 按一下「顯示」按鈕，然後選擇「在新視窗中」，即可查看實際運作的程式碼。
-

步驟 4 · 約 4 分鐘

4. 開啟序列連線

檢查是否支援 Web Serial API

首先，請檢查目前的瀏覽器是否支援 [Web Serial API](#)。如要這麼做，請檢查 `serial` 是否在 `navigator` 中。

在 `DOMContentLoaded` 事件中，將下列程式碼新增至專案：

script.js - DOMContentLoaded

```
// CODELAB: Add feature detection here.
const notSupported = document.getElementById('notSupported');
notSupported.classList.toggle('hidden', 'serial' in navigator);
```

這項檢查會確認是否支援 Web Serial。如果是，這段程式碼會隱藏「不支援 Web Serial」的橫幅。

試試看

1. 載入頁面。
2. 確認頁面未顯示紅色的橫幅，指出不支援 Web Serial。

開啟序列埠

接著，我們需要開啟序列埠。與大多數其他現代 API 相同，Web Serial API 是非同步的。這樣可避免 UI 在等待輸入時遭到封鎖，這點很重要，因為網頁隨時可能會收到序列資料，我們需要監聽這些資料。

由於電腦可能有多個序列裝置，因此瀏覽器嘗試要求通訊埠時，會提示使用者選擇要連線的裝置。

在專案中新增下列程式碼：

script.js - connect()

```
// CODELAB: Add code to request & open port here.
// - Request a port and open a connection.
port = await navigator.serial.requestPort();
// - Wait for the port to open.
await port.open({ baudRate: 9600 });
```

`requestPort` 呼叫會提示使用者要連線的裝置。呼叫 `port.open` 會開啟通訊埠。我們也需要提供與序列裝置通訊的速度。BBC micro:bit 會在 USB 對序列埠晶片和主處理器之間，使用 9600 鮑率連線。

我們也來連結「連線」按鈕，讓使用者點選時呼叫 `connect()`。

在專案中新增下列程式碼：

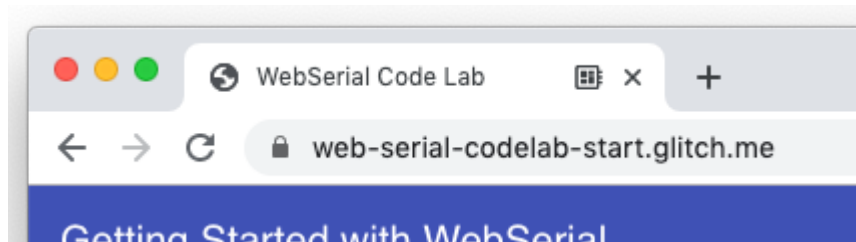
script.js - clickConnect()

```
// CODELAB: Add connect code here.
await connect();
```

試試看

現在專案已具備最基本的功能，可以開始使用。按一下「連線」按鈕後，系統會提示使用者選取要連線的序列裝置，然後連線至 micro:bit。

1. 請重新載入頁面。
2. 按一下「連結」按鈕。
3. 在「Serial Port」選擇器對話方塊中，選取 BBC micro:bit 裝置，然後按一下「Connect」。
4. 分頁上應會顯示圖示，指出您已連線至序列裝置：



提示：如果選擇器未顯示 micro:bit，請嘗試重設裝置，方法是按下背面 USB 連接埠旁的重設按鈕。

設定輸入串流，監聽序列埠的資料

建立連線後，我們需要設定輸入串流和讀取器，才能從裝置讀取資料。首先，我們會呼叫 `port.readable`，從通訊埠取得可讀取的串流。由於我們知道會從裝置收到文字，因此會透過文字解碼器傳送文字。接著，我們會取得讀取器並啟動讀取迴圈。

在專案中新增下列程式碼：

script.js - connect()

```
// CODELAB: Add code to read the stream here.
let decoder = new TextDecoderStream();
inputDone = port.readable.pipeTo(decoder.writable);
inputStream = decoder.readable;

reader = inputStream.getReader();
readLoop();
```

讀取迴圈是非同步函式，會在迴圈中執行並等待內容，不會封鎖主執行緒。當新資料抵達時，讀取器會傳回兩個屬性：`value` 和 `done` 布林值。如果 `done` 為 `true`，表示連接埠已關閉，或沒有更多資料傳入。

在專案中新增下列程式碼：

script.js - readLoop()

```
// CODELAB: Add read loop here.
while (true) {
  const { value, done } = await reader.read();
  if (value) {
    log.textContent += value + '\n';
  }
  if (done) {
    console.log('[readLoop] DONE', done);
    reader.releaseLock();
    break;
  }
}
```

試試看

現在專案可以連線至裝置，並將從裝置收到的任何資料附加至記錄檔元素。

1. 請重新載入頁面。
2. 按一下「連結」按鈕。
3. 在「Serial Port」選擇器對話方塊中，選取 BBC micro:bit 裝置，然後按一下「Connect」。
4. 您應該會看到 Espruino 標誌：



提示：在 ChromeOS 和 Linux 上，這項設定可能會損毀，因為系統服務會嘗試與裝置通訊，而作業系統會將裝置視為數據機。您可以忽略這項損毀，但畫面可能會看起來很奇怪。這也是為什麼務必先傳送 Ctrl-C 的原因。

設定輸出串流，將資料傳送至序列埠

序列通訊通常是雙向的。除了從序列埠接收資料，我們也想將資料傳送至該埠。與輸入串流相同，我們只會透過輸出串流將文字傳送至 micro:bit。

首先，請建立文字編碼器串流，並將該串流管道傳送至 `port.writable`。

```
script.js - connect()
```

```
// CODELAB: Add code setup the output stream here.
const encoder = new TextEncoderStream();
outputDone = encoder.readable.pipeTo(port.writable);
outputStream = encoder.writable;
```

透過序列埠連線至 Espruino 韌體時，BBC micro:bit 開發板會充當 JavaScript [讀取-評估-列印迴圈 \(REPL\)](#)，類似於 Node.js 殼層中的情況。接著，我們需要提供將資料傳送至串流的方法。下列程式碼會從輸出串流取得寫入器，然後使用

`write` 傳送每一行。傳送的每一行都包含換行字元 (`\n`)，告知 `micro:bit` 評估傳送的指令。

```
script.js - writeToStream()
```

```
// CODELAB: Write to output stream
const writer = outputStream.getWriter();
lines.forEach((line) => {
  console.log('[SEND]', line);
  writer.write(line + '\n');
});
writer.releaseLock();
```

如要讓系統進入已知狀態，並停止回傳我們傳送的字元，我們需要傳送 `CTRL-C` 並關閉回音。

```
script.js - connect()
```

```
// CODELAB: Send CTRL-C and turn off echo on REPL
writeToStream('\x03', 'echo(false);');
```

試試看

現在，我們的專案可以從 `micro:bit` 傳送及接收資料。請確認我們是否能正確傳送指令：

1. 請重新載入頁面。
2. 按一下「連結」按鈕。
3. 在「Serial Port」選擇器對話方塊中，選取 `BBC micro:bit` 裝置，然後按一下「Connect」。
4. 在 `Chrome` 開發人員工具中開啟「主控台」分頁，然後輸入 `writeToStream('console.log("yes")');`

頁面應會顯示類似下列的內容：

```
y
e
s
```

5. 控制 LED 矩陣

建構矩陣格線字串

如要控制 micro:bit 上的 LED 矩陣，我們需要呼叫 `show()`。這個方法會在內建的 5x5 LED 螢幕上顯示圖像。這個函式會採用二進位數字或字串。

我們會疊代核取方塊，並產生 1 和 0 的陣列，指出哪些核取方塊已勾選，哪些未勾選。接著，我們需要反轉陣列，因為核取方塊的順序與矩陣中 LED 的順序相反。接著，我們將陣列轉換為字串，並建立要傳送至 micro:bit 的指令。

script.js - sendGrid()

```
// CODELAB: Generate the grid
const arr = [];
ledCBs.forEach((cb) => {
  arr.push(cb.checked === true ? 1 : 0);
});
writeToStream(`show(0b${arr.reverse().join('')}`);
```

連結核取方塊來更新矩陣

接著，我們需要監聽核取方塊的變更，並在變更時將該資訊傳送至 micro:bit。在特徵偵測程式碼 (`// CODELAB: Add feature detection here.`) 中，加入下列程式碼行：

script.js - DOMContentLoaded

```
initCheckboxes();
```

我們也要在首次連線 micro:bit 時重設格線，讓格線顯示笑臉。系統已提供 `drawGrid()` 函式，這個函式與 `sendGrid()` 的運作方式類似，會採用 1 和 0 的陣列，並視需要勾選核取方塊。

script.js - clickConnect()

```
// CODELAB: Reset the grid on connect here.
drawGrid(GRID_HAPPY);
sendGrid();
```

試試看

現在，當網頁開啟與 micro:bit 的連線時，就會傳送笑臉。按一下核取方塊，即可更新 LED 矩陣上的顯示內容。

1. 請重新載入頁面。
 2. 按一下「連結」按鈕。
 3. 在「Serial Port」選擇器對話方塊中，選取 BBC micro:bit 裝置，然後按一下「Connect」。
 4. micro:bit LED 矩陣上應該會顯示笑臉。
 5. 變更核取方塊，在 LED 矩陣上繪製不同圖案。
-

6. 連接 micro:bit 按鈕

在 micro:bit 按鈕上新增手錶事件

micro:bit 上有兩個按鈕，分別位於 LED 矩陣的兩側。Espruino 提供 `setWatch` 函式，按下按鈕時會傳送事件/回呼。由於我們想監聽這兩個按鈕，因此會將函式設為泛型，並列印事件的詳細資料。

```
script.js - watchButton()
```

```
// CODELAB: Hook up the micro:bit buttons to print a string.
const cmd = `
  setWatch(function(e) {
    print('{ "button": "${btnId}", "pressed": ' + e.state + ' }');
  }, ${btnId}, {repeat:true, debounce:20, edge:"both"});
`;
writeToStream(cmd);
```

接著，我們需要將兩個按鈕 (在 micro:bit 板上分別命名為 BTN1 和 BTN2) 連接至裝置的序列埠。

```
script.js - clickConnect()
```

```
// CODELAB: Initialize micro:bit buttons.
watchButton('BTN1');
watchButton('BTN2');
```

試試看

除了在連線時顯示笑臉，按下 micro:bit 上的任一按鈕，也會在頁面上新增文字，指出按下的是哪個按鈕。每個字元很可能都會顯示在各自的行上。

1. 請重新載入頁面。
 2. 按一下「連結」按鈕。
 3. 在「Serial Port」選擇器對話方塊中，選取 BBC micro:bit 裝置，然後按一下「Connect」。
 4. micro:bit 的 LED 矩陣上應該會顯示笑臉。
 5. 按下 micro:bit 上的按鈕，確認頁面會附加新文字，並顯示所按按鈕的詳細資料。
-

7. 使用轉換串流剖析傳入資料

提示：如果您是串流新手，請參閱 [MDN 的 Streams API 概念](#)。本程式碼研究室僅簡單介紹串流和串流處理作業。

基本串流處理

按下 micro:bit 的其中一個按鈕時，micro:bit 會透過串流將資料傳送至序列埠。串流非常實用，但同時也是一項挑戰，因為您不一定能一次取得所有資料，而且資料可能會任意分塊。

應用程式目前會列印傳入的串流 (位於 `readLoop` 中)，但通常每個字元都會顯示在同一行，這並不是很實用。最好將串流剖析為個別行，並將每則訊息顯示為一行。

使用 `TransformStream` 轉換串流

為此，我們可以使用轉換串流 (`TransformStream`)，以便剖析傳入的串流並傳回剖析的資料。轉換串流可位於串流來源 (本例為 micro:bit) 與串流取用者 (本例為 `readLoop`) 之間，並在最終取用前套用任意轉換。這就像組裝線：當小工具沿著組裝線移動時，組裝線上的每個步驟都會修改小工具，因此小工具抵達最終目的地時，就會成為功能齊全的小工具。

詳情請參閱 [MDN 的 Streams API 概念](#)。

使用 `LineBreakTransformer` 轉換串流

讓我們建立 `LineBreakTransformer` 類別，該類別會接收串流，並根據換行符 (`\r\n`) 將串流分塊。這個類別需要 `transform` 和 `flush` 這兩個方法。每當串流收到新資料時，系統就會呼叫 `transform` 方法。它可以將資料加入佇列，也可以儲存資料以供日後使用。串流關閉時會呼叫 `flush` 方法，並處理尚未處理的任何資料。

在 `transform` 方法中，我們會將新資料新增至 `container`，然後檢查 `container` 中是否有任何換行符。如果有的話，請將其分割成陣列，然後逐行疊代，呼叫 `controller.enqueue()` 傳送剖析的行。

```
script.js - LineBreakTransformer.transform()
```

```
// CODELAB: Handle incoming chunk
this.container += chunk;
const lines = this.container.split('\r\n');
this.container = lines.pop();
lines.forEach(line => controller.enqueue(line));
```

串流關閉時，我們會使用 `enqueue` 清除容器中所有剩餘資料。

```
script.js - LineBreakTransformer.flush()
```

```
// CODELAB: Flush the stream.
controller.enqueue(this.container);
```

最後，我們需要透過新的 `LineBreakTransformer` 管道傳送傳入的串流。原始輸入串流只透過 `TextDecoderStream` 管道傳輸，因此我們需要新增 `pipeThrough`，透過新的 `LineBreakTransformer` 管道傳輸。

```
script.js - connect()
```

```
// CODELAB: Add code to read the stream here.
let decoder = new TextDecoderStream();
inputDone = port.readable.pipeTo(decoder.writable);
inputStream = decoder.readable
    .pipeThrough(new TransformStream(new LineBreakTransformer()));
```

試試看

現在按下 micro:bit 按鈕時，系統應該會在一行中傳回列印的資料。

1. 請重新載入頁面。
2. 按一下「連結」按鈕。
3. 在「Serial Port」選擇器對話方塊中，選取 BBC micro:bit 裝置，然後按一下「Connect」。
4. micro:bit LED 矩陣上應該會顯示笑臉。
5. 按下 micro:bit 上的按鈕，確認您看到類似下列內容：

```
{ "button": "BTN1", "pressed": true }
{ "button": "BTN1", "pressed": false }
```

使用 `JSONTransformer` 轉換串流

我們可以在 `readLoop` 中嘗試將字串剖析為 JSON，但我們改為建立非常簡單的 JSON 轉換器，將資料轉換為 JSON 物件。如果資料不是有效的 JSON，只要傳回傳入的內容即可。

```
script.js - JSONTransformer.transform
```

```
// CODELAB: Attempt to parse JSON content
try {
    controller.enqueue(JSON.parse(chunk));
} catch (e) {
    controller.enqueue(chunk);
}
```

接著，在串流通過 `LineBreakTransformer` 後，透過 `JSONTransformer` 管道傳輸串流。由於我們知道 JSON 只會在一行中傳送，因此可以簡化 `JSONTransformer`。

```
script.js - connect
```

```
// CODELAB: Add code to read the stream here.
let decoder = new TextDecoderStream();
inputDone = port.readable.pipeTo(decoder.writable);
inputStream = decoder.readable
    .pipeThrough(new TransformStream(new LineBreakTransformer()))
    .pipeThrough(new TransformStream(new JSONTransformer()));
```

試試看

現在按下 micro:bit 按鈕時，頁面應該會顯示 `[object Object]`。

1. 請重新載入頁面。
2. 按一下「連結」按鈕。
3. 在「Serial Port」選擇器對話方塊中，選取 BBC micro:bit 裝置，然後按一下「Connect」。
4. micro:bit LED 矩陣上應該會顯示笑臉。
5. 按下 micro:bit 上的按鈕，確認您看到類似下列內容：

回應按鈕動作

如要回應 micro:bit 按鈕按下事件，請更新 `readLoop`，檢查收到的資料是否為具有 `button` 屬性的 `object`。然後呼叫 `buttonPushed` 來處理按鈕推送。

`script.js - readLoop()`

```
const { value, done } = await reader.read();
if (value && value.button) {
  buttonPushed(value);
} else {
  log.textContent += value + '\n';
}
```

按下 micro:bit 按鈕時，LED 矩陣上的顯示內容應會變更。請使用下列程式碼設定矩陣：

`script.js - buttonPushed()`

```
// CODELAB: micro:bit button press handler
if (butEvt.button === 'BTN1') {
  divLeftBut.classList.toggle('pressed', butEvt.pressed);
  if (butEvt.pressed) {
    drawGrid(GRID_HAPPY);
    sendGrid();
  }
  return;
}
if (butEvt.button === 'BTN2') {
  divRightBut.classList.toggle('pressed', butEvt.pressed);
  if (butEvt.pressed) {
    drawGrid(GRID_SAD);
    sendGrid();
  }
}
```

試試看

現在按下其中一個 micro:bit 按鈕時，LED 矩陣應該會變成笑臉或哭臉。

1. 請重新載入頁面。
2. 按一下「連結」按鈕。
3. 在「Serial Port」選擇器對話方塊中，選取 BBC micro:bit 裝置，然後按一下「Connect」。
4. micro:bit 的 LED 矩陣上應該會顯示笑臉。
5. 按下 micro:bit 上的按鈕，確認 LED 矩陣是否變更。

步驟 8 · 約 1 分鐘

8. 關閉序列埠

最後一個步驟是連結中斷連線功能，在使用者完成作業時關閉通訊埠。

使用者點選「連線」/「中斷連線」按鈕時關閉通訊埠

當使用者點選「連線」/「中斷連線」按鈕時，我們需要關閉連線。如果通訊埠已開啟，請呼叫 `disconnect()` 並更新 UI，指出網頁已與序列裝置中斷連線。

script.js - `clickConnect()`

```
// CODELAB: Add disconnect code here.
if (port) {
  await disconnect();
  toggleUIConnected(false);
  return;
}
```

關閉串流和通訊埠

在 `disconnect` 函式中，我們需要關閉輸入串流、輸出串流和通訊埠。如要關閉輸入串流，請呼叫 `reader.cancel()`。對 `cancel` 的呼叫是非同步的，因此我們需要使用 `await` 等待呼叫完成：

script.js - `disconnect()`

```
// CODELAB: Close the input stream (reader).
if (reader) {
  await reader.cancel();
  await inputDone.catch(() => {});
  reader = null;
  inputDone = null;
}
```

如要關閉輸出串流，請取得 `writer`、呼叫 `close()`，然後等待 `outputDone` 物件關閉：

script.js - `disconnect()`

```
// CODELAB: Close the output stream.
if (outputStream) {
  await outputStream.getWriter().close();
  await outputDone;
  outputStream = null;
  outputDone = null;
}
```

最後，關閉序列埠並等待關閉：

script.js - `disconnect()`


```
// CODELAB: Close the port.  
await port.close();  
port = null;
```

試試看

現在，您可以隨意開啟及關閉序列埠。

1. 請重新載入頁面。
 2. 按一下「連結」按鈕。
 3. 在「Serial Port」選擇器對話方塊中，選取 BBC micro:bit 裝置，然後按一下「Connect」。
 4. micro:bit LED 矩陣上應該會顯示笑臉
 5. 按下「Disconnect」(中斷連線) 按鈕，確認 LED 矩陣關閉，且主控台中沒有錯誤。
-

步驟 9 · 約 0 分鐘

9. 恭喜

恭喜！您已成功建構第一個使用 Web Serial API 的網頁應用程式。

請密切關注 <https://goo.gle/fugu-api-tracker>，瞭解 Web Serial API 的最新消息，以及 Chrome 團隊正在開發的所有其他令人期待的全新網頁功能。
